

Java Type Conversion

Implicit • Explicit • Complete Reference

Unit 3 – Lecture 3

By Sourabh Gorai | Channel: NeverStop

Class 9 | Java Programming

■ Topic	■ What You Will Learn
What is Type Conversion?	Changing a value from one data type to another
Implicit (Widening)	Java converts automatically — no data loss
Explicit (Narrowing)	Programmer manually converts — data may be lost
Type Casting Syntax	(targetType) value — the cast operator
String ↔ Number	parseInt, parseDouble, toString, valueOf
Common Mistakes & Tips	Overflow, precision loss, NumberFormatException

1. What is a Data Type?

In Java, every variable must have a **data type** telling the compiler what kind of value it stores and how much memory to use. Java has two categories:

Category	Types	Size
Primitive	byte, short, int, long	1, 2, 4, 8 bytes
Primitive	float, double	4, 8 bytes
Primitive	char	2 bytes
Primitive	boolean	1 bit
Non-Primitive	String, Arrays, Classes	varies

■ Type Conversion means converting a value of one data type into another. Java supports both automatic and manual conversion.

2. Java's Type Hierarchy (Widening Order)

Java primitive types have a natural **hierarchy from smaller to larger**. Moving **UP** (widening) is always safe. Moving **DOWN** (narrowing) requires a manual cast.



Also valid: `char` → `int` → `long` → `float` → `double`

3. Implicit Type Conversion (Widening)

3.1 What is it?

When Java **automatically** converts a smaller data type into a larger data type — without any instruction from the programmer — it is called **Implicit Type Conversion** or **Widening Conversion**.

3.2 Rules

- ◆ Happens automatically — the compiler handles it with no extra code.
- ◆ No data is lost (the smaller value fits perfectly inside the larger type).
- ◆ Only works going from a **smaller** to a **larger** type.
- ◆ No cast operator `()` is needed.
- ◆ Also called **Automatic Type Promotion**.

3.3 Example Code

```

public class WideningDemo {
public static void main(String[] args) {
byte b = 10;
short s = b; // byte → short (auto)
int i = s; // short → int (auto)
long l = i; // int → long (auto)
float f = l; // long → float (auto)
double d = f; // float → double (auto)
System.out.println("byte = " + b);
System.out.println("short = " + s);
System.out.println("int = " + i);
System.out.println("long = " + l);
System.out.println("float = " + f);
System.out.println("double = " + d);
}
}

```

Output:

```

byte = 10
short = 10
int = 10
long = 10
float = 10.0
double = 10.0

```

3.4 char → int (Special Case)

A **char** value is stored internally as a Unicode number. When implicitly converted to **int**, you get its ASCII/Unicode value.

```

char ch = 'A';
int num = ch; // Implicit: char → int
// num is now 65 (ASCII value of 'A')
System.out.println(num); // Output: 65

```

■ ASCII quick ref: 'A'=65 'Z'=90 'a'=97 'z'=122 '0'=48 '9'=57. All chars have a fixed Unicode number.

3.5 Expression Promotion (Arithmetic)

Inside arithmetic expressions, Java automatically promotes smaller types to prevent overflow. Key rules:

- **byte** and **short** are always promoted to **int** before arithmetic.
- If one operand is **long**, the whole expression becomes **long**.
- If one operand is **float**, the whole expression becomes **float**.
- If one operand is **double**, the whole expression becomes **double**.

```

byte a = 40;
byte b = 50;
// byte result = a + b; ← COMPILER ERROR – a+b is promoted to int!

```

```
int result = a + b; // ← CORRECT  
System.out.println(result); // 90
```

■■ byte + byte = int in Java! You cannot store the result back into a byte without an explicit cast.

4. Explicit Type Conversion (Narrowing / Casting)

4.1 What is it?

When you **manually** convert a larger data type into a smaller one, it is called **Explicit Type Conversion** or **Narrowing Conversion**. Java does NOT do this automatically because **data can be lost**. The programmer must use the **cast operator**.

4.2 Cast Operator Syntax

```
(targetType) value
// Example:
double d = 9.99;
int i = (int) d; // ← explicit cast
System.out.println(i); // Output: 9 (decimal part is LOST)
```

■ The decimal part is TRUNCATED, not rounded. 9.99 becomes 9, NOT 10. Use Math.round() if you need rounding.

4.3 Step-by-Step: How to Cast

Step 1	Identify the target type you want to convert to.
Step 2	Write the target type inside parentheses before the value: (int)
Step 3	Assign the result to a variable of the target type.
Step 4	Verify that data loss is acceptable in your program.

4.4 More Examples

```
public class NarrowingDemo {
    public static void main(String[] args) {
        // double → int (decimal part cut off)
        double pi = 3.14159;
        int i = (int) pi;
        System.out.println(i); // 3

        // long → int (overflow if value too big)
        long bigNum = 1234567890123L;
        int small = (int) bigNum;
        System.out.println(small); // garbage value!

        // double → byte (overflow wraps around)
        double d = 257.75;
        byte by = (byte) d;
        System.out.println(by); // 1
    }
}
```

```
// int → char
int code = 65;
char ch = (char) code;
System.out.println(ch); // A
}
}
```

4.5 Data Loss in Narrowing — Why it Happens

Conversion	What is Lost?	Example
double → int	Fractional (decimal) part	9.99 → 9
double → float	Precision (some digits)	3.141592653 → 3.1415927
int → byte	High-order bits (overflow)	300 → 44
long → int	High-order bits (overflow)	Large value → wrong
int → char	Negative/out-of-range vals	65 → 'A' (safe here)

5. String ↔ Number Conversion

String is not a primitive type — it is a class. Converting between String and numbers requires **methods**, not cast operators.

5.1 Number → String

```
// Method 1: String.valueOf()
int num = 42;
String s1 = String.valueOf(num); // "42"

// Method 2: Integer.toString()
String s2 = Integer.toString(num); // "42"

// Method 3: Concatenation with empty string
String s3 = num + ""; // "42"

// Works for all primitive types:
double d = 3.14;
String sd = String.valueOf(d); // "3.14"
```

5.2 String → Number (Parsing)

```
// String → int
String s = "123";
int num = Integer.parseInt(s); // 123

// String → double
String ds = "3.14";
double d = Double.parseDouble(ds); // 3.14

// String → long
long l = Long.parseLong("9876543210");

// String → float
float f = Float.parseFloat("2.5");

// String → byte / short
byte b = Byte.parseByte("100");
short sh = Short.parseShort("500");
```

■ If the String is not a valid number (e.g. "abc"), `parseInt()` throws a `NumberFormatException` at runtime. Always validate the string first.

5.3 Quick Reference Table

Task	Method	Example
String → int	<code>Integer.parseInt()</code>	<code>parseInt("10") → 10</code>
String → double	<code>Double.parseDouble()</code>	<code>parseDouble("3.5") → 3.5</code>
String → long	<code>Long.parseLong()</code>	<code>parseLong("999") → 999L</code>
String → float	<code>Float.parseFloat()</code>	<code>parseFloat("1.5") → 1.5f</code>

int → String	String.valueOf()	valueOf(10) → "10"
--------------	------------------	--------------------

6. Implicit vs Explicit — Quick Comparison

Feature	Implicit (Widening)	Explicit (Narrowing)
Done by	Java Compiler	Programmer
Keyword needed?	No	Yes — () cast operator
Direction	Smaller → Larger	Larger → Smaller
Data Loss?	No	Yes (possible)
Safe?	Always safe	Risky — check your values
Also called	Upcasting / Promotion	Downcasting / Casting
Example	<code>int i=5; double d=i;</code>	<code>double d=5.9; int i=(int)d;</code>

7. Common Mistakes & How to Avoid Them

■ Mistake 1: Forgetting to cast in expressions

Wrong:

```
byte a = 10; byte b = 20; byte c = a + b; // ERROR
```

Fix:

```
byte c = (byte)(a + b); // CORRECT
```

■ Mistake 2: Expecting rounding, getting truncation

Wrong:

```
double d = 9.9; int i = (int) d; // i = 9, NOT 10
```

Fix:

```
int i = (int) Math.round(d); // i = 10 (if you want rounding)
```

■ Mistake 3: Overflow in narrowing

Wrong:

```
int x = 300; byte b = (byte) x; // b = 44, not 300!
```

Fix:

```
// Always check range: byte is -128 to 127 only
```

■ Mistake 4: Parsing an invalid String

Wrong:

```
int n = Integer.parseInt("abc"); // NumberFormatException!
```

Fix:

```
// Make sure String contains a valid number before parsing
```


8. Practice Questions

Q1.	What is the output? <code>int i = 10; double d = i; System.out.println(d);</code>
Q2.	What is the output? <code>double d = 7.8; int i = (int) d; System.out.println(i);</code>
Q3.	What is the output? <code>char c = 'Z'; int n = c; System.out.println(n);</code>
Q4.	Find the error: <code>byte a = 100; byte b = 50; byte c = a + b;</code>
Q5.	What is the output? <code>String s = "25"; int n = Integer.parseInt(s) + 5; System.out.println(n);</code>
Q6.	What type does 'byte + byte' return in Java?
Q7.	Write code to convert int 97 to its character equivalent.
Q8.	Will this compile? Why? <code>long l = 1000; int i = l;</code>

Answers

A1.	10.0 — implicit widening: int is promoted to double
A2.	7 — explicit narrowing: the .8 decimal is truncated
A3.	90 — 'Z' has ASCII/Unicode value 90
A4.	Compile Error: byte+byte = int; fix: byte c = (byte)(a+b);
A5.	30 — parseInt("25")=25, then 25+5=30
A6.	int — byte operands are always promoted to int in expressions
A7.	<code>int i = 97; char c = (char) i; → c = 'a'</code>
A8.	No — long is larger than int; needs explicit cast: <code>int i = (int) l;</code>

9. Summary Cheat Sheet

Concept	One-liner
Implicit Conversion	Java auto-converts smaller → larger (safe, no cast needed)
Explicit Conversion	Programmer casts larger → smaller using (type) — may lose data
Widening Order	byte → short → int → long → float → double
Cast Syntax	(targetType) variable e.g. (int) 3.9 → 3
Expression Promotion	byte / short → int automatically in any expression
char → int	Gives the Unicode / ASCII value (implicit, automatic)
int → char	Gives the character for that code (explicit cast needed)
String → int	Integer.parseInt(str)
int → String	String.valueOf(num) or num + ""
NumberFormatException	Thrown when parsing a non-numeric String like "abc"
Overflow in narrowing	Value wraps around (e.g. 300 stored as byte → 44)
Truncation	Decimal part is CUT, NOT rounded (9.9 → 9)

■ Remember: WIDENING = safe + automatic (small → big). NARROWING = risky + manual (big → small). Always think about whether your data will survive the conversion!